Manual de Testing — RestaurantHub API con Postman

Guía para probar manualmente la API de RestaurantHub usando Postman, incluyendo qué datos deben existir previamente en la base de datos.

Tabla de contenidos

1. Prerrequisitos
2. Punto clave: qué hay que precargar en la DB
3. Configuración de Postman
4. Flujo de pruebas recomendado
5. Detalle de cada bloque de pruebas
6. Errores comunes y cómo interpretarlos
7. Checklist final

---

1. Prerrequisitos

- **Postman** instalado (o su versión web).
- La **API corriendo localmente** (`npm run dev` o equivalente), normalmente en `http://localhost:3000/api`.
- **MongoDB** corriendo y accesible (local o Atlas), con la base de datos vacía o de prueba.
- Acceso a **mongosh** o **MongoDB Compass** para cargar datos iniciales (ver sección 2).

---

2. Punto clave: qué hay que precargar en la DB

Este es el punto más importante antes de empezar a testear: **la API no expone endpoints para crear Mesas (`Table`) ni Categorías (`Category`)**. Solo existen endpoints para *leer* y *actualizar* mesas (`GET /api/tables`, `PUT /api/tables/:id/status`, `PUT /api/tables/:id/assign-order`), pero ninguno para darlas de alta. Lo mismo pasa con las categorías: se referencian en `MenuItem.category`, pero no hay un módulo `/api/categories`.

Esto significa que, **si no cargás mesas manualmente en Mongo, no vas a poder probar reservas ni órdenes**, porque ambos flujos dependen de que existan mesas disponibles.

2.1. Datos que sí se generan solos (vía API)

No hace falta precargarlos, se crean probando los endpoints:

Dato	Cómo se genera
Usuarios (`User`)	`POST /api/auth/register`
Platos (`MenuItem`)	`POST /api/menu/items` (con usuario admin)
Reservas (`Reservation`)	`POST /api/reservations`
Órdenes (`Order`)	`POST /api/orders`
Transacciones (`Transaction`)	`POST /api/transactions`
Caja (`CashRegister`)	`POST /api/cash-register/open`

2.2. Datos que hay que cargar a mano en Mongo

Dato	Por qué	Mínimo recomendado
`Table` (mesas)	No existe endpoint de creación	3-4 mesas con distinta capacidad y ubicación
`Category` (categorías)	No existe endpoint de creación (si tu implementación las valida contra la colección)	2-3 categorías básicas: `principales`, `ensaladas`, `bebidas`

2.3. Ejemplo de carga con mongosh

Conectate a la base y ejecutá:

```javascript
```

```
use restauranthub

// Mesas
db.tables.insertMany([
  { number: 1, capacity: 2, location: "interior", status: "free", currentOrderI
  { number: 2, capacity: 4, location: "interior", status: "free", currentOrderI
  { number: 5, capacity: 6, location: "ventana", status: "free", currentOrderId
  { number: 8, capacity: 4, location: "patio", status: "free", currentOrderId:
])

// Categorías
db.categories.insertMany([
  { name: "principales", description: "Platos principales", icon: "🍽️", created
  { name: "ensaladas", description: "Ensaladas", icon: "🥗", createdAt: new Dat
  { name: "bebidas", description: "Bebidas", icon: "🥤", createdAt: new Date(),
])
```

> Ajustá los nombres de colecciones (`tables`, `categories`) si tu implementación usa otros nombres. Si usás Mongoose, revisá el `modelName` de cada schema.

2.4. Alternativa sin tocar Mongo directamente

Si preferís no usar mongosh, podés cargar los mismos documentos con **MongoDB Compass** (interfaz gráfica): conectar a la base, entrar a la colección `tables`/`categories` y usar "Insert Document" pegando el JSON equivalente.

---

3. Configuración de Postman

3.1. Crear una Collection

1. `New` → `Collection` → nombrarla **RestaurantHub API**.
2. Dentro de la collection, crear una carpeta por módulo: `Auth`, `Menu`, `Reservations`, `Orders`, `Tables`, `Transactions`, `Stats`.

3.2. Crear un Environment

`New` → `Environment` → **RestaurantHub Local**, con estas variables (dejar el "Current value" vacío salvo `baseUrl`):

Variable	Initial value	Descripción
`baseUrl`	`http://localhost:3000/api`	URL base de la API
`adminToken`	*(vacío)*	Token JWT del usuario admin
`clientToken`	*(vacío)*	Token JWT del usuario cliente
`refreshToken`	*(vacío)*	Refresh token
`tableId`	*(vacío)*	ID de una mesa precargada
`menuItemId`	*(vacío)*	ID de un plato creado
`reservationId`	*(vacío)*	ID de una reserva creada
`orderId`	*(vacío)*	ID de una orden creada
`cashRegisterId`	*(vacío)*	ID de la caja abierta

Seleccioná este environment arriba a la derecha antes de empezar.

3.3. Autocompletar variables con scripts (Tests tab)

En cada request que devuelve un `id` o `token`, agregá en la pestaña **Tests** un script que lo guarde automáticamente. Ejemplo para `POST /api/auth/login`:

```javascript
const res = pm.response.json();
if (res.success) {
  pm.environment.set("adminToken", res.data.token);
  pm.environment.set("refreshToken", res.data.refreshToken);
}
```

Ejemplo para `POST /api/menu/items`:

```javascript
const res = pm.response.json();
if (res.success) {
  pm.environment.set("menuItemId", res.data.id);
}
```

Así, en las siguientes requests podés usar `{{adminToken}}`, `{{menuItemId}}`, etc. en headers y URLs sin copiar/pegar manualmente.

3.4. Header de autenticación

Para los endpoints que requieren token, en la pestaña **Authorization** de cada request elegí `Bearer Token` y usá `{{adminToken}}` o `{{clientToken}}` según corresponda. Alternativamente, agregar el header manualmente:

```
Authorization: Bearer {{adminToken}}
```

---

4. Flujo de pruebas recomendado

Seguí este orden — cada bloque depende de que el anterior haya funcionado:

```
1. Precargar mesas y categorías en Mongo   (sección 2)
2. Auth        → registrar admin y cliente, login de ambos
3. Menu        → crear platos (admin)
4. Tables      → listar mesas, guardar un tableId
5. Reservations→ consultar slots, crear reserva
6. Orders      → crear orden usando tableId y menuItemId
7. Tables      → asignar orden a mesa, cambiar estado
8. Transactions→ abrir caja, registrar pago de la orden, cerrar caja
9. Stats       → consultar ventas, top items, ocupación, reportes
```

---

5. Detalle de cada bloque de pruebas

5.1. Auth

#	Request	Notas
1	`POST /api/auth/register`	Body con `"role": "admin"` → crear un admin
2	`POST /api/auth/register`	Body con `"role": "client"` → crear un cliente
3	`POST /api/auth/login`	Con el admin → guardar `adminToken`
4	`POST /api/auth/login`	Con el cliente → guardar `clientToken`
5	`POST /api/auth/refresh-token`	Usar `{{refreshToken}}`, verificar nuevo token

#	Request	Notas
6	`POST /api/auth/logout`	Con `{{clientToken}}`

Casos de error a probar:

- Registrar el mismo email dos veces → `400 EMAIL_EXISTS`.
- Login con contraseña incorrecta → `401 INVALID_CREDENTIALS`.

5.2. Menú

#	Request	Notas
1	`POST /api/menu/items`	Con `{{adminToken}}`, usar `category` acorde a lo precargado (ej. `"principales"`) → guardar `menuItemId`
2	`GET /api/menu/items`	Sin token, verificar que aparece el plato creado
3	`GET /api/menu/items/:id`	Usar `{{menuItemId}}`
4	`PUT /api/menu/items/:id`	Cambiar precio o disponibilidad
5	`POST /api/menu/items/:id/upload-image`	Body `form-data`, key `image`, tipo `File`
6	`DELETE /api/menu/items/:id`	Probar al final, para no romper pruebas de Orders que dependen del plato

Casos de error a probar:

- Crear un plato con `{{clientToken}}` (no admin) → `403 FORBIDDEN`.
- Consultar un `id` inexistente → `404 ITEM_NOT_FOUND`.

5.3. Mesas

#	Request	Notas
1	`GET /api/tables`	Con `{{adminToken}}` → copiar un `id` de una mesa precargada a `tableId`

#	Request	Notas
2	`GET /api/tables/availability`	Query `date`, `time`, `guestCount`
3	`PUT /api/tables/:id/status`	Cambiar a `"cleaning"` y volver a `"free"`

5.4. Reservas

#	Request	Notas
1	`POST /api/reservations/available-slots`	Body con `date` y `guestCount`
2	`POST /api/reservations`	Con `{{clientToken}}` → guardar `reservationId`
3	`GET /api/reservations`	Con `{{clientToken}}` (ve solo las propias) y con `{{adminToken}}` (ve todas)
4	`GET /api/reservations/:id`	Usar `{{reservationId}}`
5	`PUT /api/reservations/:id/status`	Con `{{adminToken}}`, cambiar a `"confirmed"`
6	`DELETE /api/reservations/:id`	Cancelar (probar al final del bloque)

Caso de error a probar: reservar todas las mesas del mismo horario/capacidad y volver a intentar → `409 NO_TABLES_AVAILABLE`.

5.5. Órdenes

#	Request	Notas
1	`POST /api/orders`	Con `{{clientToken}}`, `tableId: {{tableId}}`, items con `{{menuItemId}}` → guardar `orderId`
2	`GET /api/orders`	Verificar listado
3	`GET /api/orders/:id`	Usar `{{orderId}}`, revisar el `timeline`
4	`PUT /api/orders/:id/status`	Ir avanzando: `confirmed` → `in-preparation` → `ready` → `delivered`

5.6. Asignación de mesa a la orden

#	Request	Notas
1	`PUT /api/tables/:id/assign-order`	`tableId` = `{{tableId}}`, body con `{{orderId}}`
2	`GET /api/tables`	Confirmar que la mesa quedó en estado `occupied`

5.7. Transacciones / Caja

#	Request	Notas
1	`POST /api/cash-register/open`	Con `{{adminToken}}` → guardar `cashRegisterId`
2	`POST /api/transactions`	`orderId: {{orderId}}`, `amount` = total de la orden
3	`GET /api/transactions`	Verificar que aparece la transacción
4	`POST /api/cash-register/close`	Con `finalAmount` acorde a lo cobrado

5.8. Estadísticas

#	Request	Notas
1	`GET /api/stats/sales`	`?period=day`
2	`GET /api/stats/top-items`	`?limit=5`
3	`GET /api/stats/occupancy`	`?date=<hoy>`
4	`GET /api/stats/reports`	Rango `startDate`/`endDate` que cubra las pruebas realizadas

6. Errores comunes y cómo interpretarlos

Código	Causa típica en testing	Solución
`401 INVALID_TOKEN`	El token expiró o no se seteó la	Volver a hacer login o usar `refresh-`

Código	Causa típica en testing	Solución
	variable de environment	`token`
`403 FORBIDDEN`	Se está usando `clientToken` en un endpoint de admin	Usar `{{adminToken}}`
`404 ITEM_NOT_FOUND`	El `id` no existe o pertenece a otra colección	Verificar que la variable (`menuItemId`, `orderId`, etc.) esté seteada correctamente
`409 NO_TABLES_AVAILABLE`	No hay mesas precargadas con capacidad suficiente, o ya están todas reservadas/ocupadas	Cargar más mesas en Mongo (sección 2.3) o liberar mesas con `PUT /api/tables/:id/status`
`409 TABLE_OCCUPIED`	Se intenta asignar una orden a una mesa ya ocupada	Cambiar el estado de la mesa a `free` primero
Respuesta vacía / conexión rechazada	El servidor no está corriendo o `baseUrl` está mal	Verificar `npm run dev` y la variable `baseUrl` del environment

7. Checklist final

- [ ] Mesas cargadas manualmente en Mongo
- [ ] Categorías cargadas manualmente en Mongo (si tu implementación las requiere)
- [ ] Environment de Postman creado y seleccionado
- [ ] Usuario admin y cliente registrados y logueados
- [ ] Al menos un plato creado
- [ ] Reserva y orden creadas de punta a punta
- [ ] Ciclo completo de caja (abrir → transacción → cerrar) probado
- [ ] Endpoints de estadísticas devuelven datos coherentes con lo probado